

Enhanced LLM Reasoning by Optimizing Reward Functions with Search-Driven Reinforcement Learning

Arash Ahmadi, Sarah Sharif, Yaser (Mike) Banad

Abstract—Mathematical reasoning is a key benchmark for large language models. Reinforcement learning is a standard post-training mechanism for improving the reasoning capabilities of large language models, yet performance remains sensitive to the design of the reward function that drives policy optimization. This paper introduces a search-driven framework that treats the reward specification itself as an object of optimization. The setting of interest is one in which the base model is held fixed and the reward specification is the primary remaining design lever. Candidate reward functions are generated by a frontier language model, validated automatically, screened through 500-step Group Relative Policy Optimization (GRPO) training runs on a Llama-3.2-3B-Instruct base model with Low-Rank Adaptation (LoRA), and ranked by F1 on the GSM8K test set. Ranked summaries from prior rounds are then fed back into the next round of generation. Over five rounds, the search produces 50 candidate rewards. The mean F1 rises from 0.596 in Round 1 to 0.632 in Round 5, and the top individual reward reaches F1=0.787. Seven ensemble configurations of top-ranked rewards are evaluated. The best ensemble achieves F1=0.795 (95% bootstrap CI [0.756, 0.832]) and accuracy 0.660 [0.635, 0.686], a 0.19 absolute F1 gain over a base-rewards-only GRPO baseline (F1=0.609). Pairwise McNemar tests with Bonferroni correction show all five-or-more-reward configurations are statistically indistinguishable at $\alpha = 0.05/21$. A three-seed re-training of the best ensemble yields F1 of 0.785 ± 0.013 . A randomly drawn 5-reward control collapses to F1=0.047, which shows that the ranked-feedback loop, not the additive signal of having more rewards, drives the gain.

Impact Statement—Reinforcement learning is increasingly used to improve reasoning in AI language models, but designing effective reward functions remains a manual and error-prone process that typically requires expert intuition and trial-and-error. The framework presented here automates reward discovery and reduces human design effort while keeping each reward as readable Python code that practitioners can audit, modify, and combine. The pipeline runs on a single consumer-grade GPU in roughly 40 hours, which puts the method inside the reach of academic and small-industry compute budgets. The discovered rewards measurably improve mathematical-reasoning quality on a fixed base model and produce fewer incorrect answers when the model does respond. The same recipe transfers to other reasoning domains such as code generation and scientific problem solving and offers a practical alternative to expensive reward-model training for aligning language models with target reasoning behaviors. All code and discovered reward functions are released

at <https://github.com/INQUIRELAB/search-reward-rl>.

Index Terms—Search-driven reward optimization, large language models, mathematical reasoning, reinforcement learning, reward function design

I. INTRODUCTION

MATHEMATICAL reasoning has become one of the clearest stress tests for large language models (LLMs) because success requires more than fluent text generation. A capable model must preserve intermediate consistency, carry quantities over multiple steps, and produce a final answer that is both correct and precisely stated. Prompting-based methods such as chain-of-thought prompting and self-consistency improved arithmetic and symbolic reasoning substantially, and benchmarks such as GSM8K made these gains measurable at scale [1], [2], [3]. However, prompting changes how a model is queried rather than what the model is rewarded to do during training.

Reinforcement learning (RL) has re-emerged as a central post-training mechanism for aligning LLMs with desired behaviors. Modern policy optimization methods such as Proximal Policy Optimization (PPO) made RL stable enough for large-scale language applications [4]. In language modeling, the same logic underlies reinforcement learning from human feedback (RLHF), where human or model-derived preferences are converted into training signals that push a policy beyond imitation of static demonstrations [5], [6], [7]. Direct Preference Optimization (DPO) shows that preference optimization can sometimes bypass explicit online RL, but it does not remove the underlying question of what constitutes a useful reward or preference signal [8].

That question becomes especially important in reasoning tasks. A reward that scores only the final answer is easy to compute when ground-truth labels are available, but it is also coarse: it cannot reliably distinguish a correct answer reached through faithful reasoning from one reached by shallow pattern matching. A reward that evaluates intermediate reasoning steps offers richer credit assignment, yet it is much harder to design, annotate, and validate at scale [9], [10]. Reward models themselves vary in quality and robustness [11]. Reasoning performance is often limited not only by the base model or the policy optimizer, but also by the quality of the reward specification. The broader RL literature has long recognized that reward design is a bottleneck. Human-written rewards are often proxies for the behavior a practitioner actually wants, and poorly chosen proxies can induce reward hacking, shortcut

A. Ahmadi, S. Sharif, and Y. M. Banad are with the School of Electrical and Computer Engineering, University of Oklahoma, Norman, OK 73019 USA (e-mail: arash.ahmadi@ou.edu; s.sh@ou.edu; bana@ou.edu). All authors are members of the Intelligent Neuromorphic and Quantum Understanding for Innovative Research and Engineering (INQUIRE) Laboratory, University of Oklahoma, Norman, OK, USA. *Corresponding author:* Y. M. Banad.

Code, generated reward functions, and experimental artifacts are available at <https://github.com/INQUIRELAB/search-reward-rl>.

learning, or brittle behavior [12]. This has motivated work on intrinsic rewards, meta-learned objectives, and automated RL systems that treat the objective itself as something to be optimized rather than assumed fixed [13], [14], [15].

Recent advances in LLMs have made reward design more programmable. Modern language models can synthesize, critique, and revise executable code, and they can participate directly in reward construction. Prior work has used LLMs as proxy reward functions, transformed language instructions into reward parameters, generated dense reward code from task descriptions, and iteratively improved reward programs through performance feedback [16], [17], [18], [19], [20], [21]. The common lesson is that reward design can be partially outsourced to a generative model when it is embedded in an iterative feedback loop. In parallel, reasoning-focused post-training has shown that RL can materially improve mathematical and logical reasoning. Group Relative Policy Optimization (GRPO), introduced for DeepSeekMath, replaces the separate critic used in PPO with group-relative normalization of sampled outputs. This change reduces memory overhead and improves mathematical reasoning [22]. DeepSeek-R1 further demonstrated that RL can elicit stronger reasoning behaviors such as self-correction at scale [23]. Yet in most of this literature the reward is still treated as fixed infrastructure: a handcrafted rule set, a trained reward model, or a single verifier.

This paper studies a complementary question: whether the reward specification used during RL post-training can itself be the object of search. The setting of interest is one in which the base model is held fixed by the practitioner and the reward function becomes the dominant remaining design variable. The framework introduced here treats the reward function as an object of search. Candidate reward functions are generated, validated, screened through GRPO training runs, ranked by downstream reasoning performance, and then fed back into the next round of reward generation. Rather than asking only which policy performs best under a fixed reward, this work asks whether the reward can itself be progressively improved as a sequence of competing hypotheses about what good mathematical reasoning looks like.

The research makes four contributions. First, it formulates reward search for language-model reasoning as an iterative generate-validate-train-rank loop over executable reward functions. Second, it applies this loop under GRPO, a reasoning-relevant and computationally efficient policy optimizer. Third, it examines whether multiple discovered rewards can be combined into stronger ensemble signals than any individual reward alone. Fourth, it provides an empirical analysis, including bootstrap confidence intervals, pairwise McNemar tests with multiple-comparison correction, and a reward-hacking audit, of how the discovered rewards trade off reasoning quality, format compliance, and computational cost on GSM8K.

II. RELATED WORK

A. Mathematical Reasoning in LLMs

Mathematical reasoning became a focal evaluation domain for LLMs because it exposes failures that remain hidden

in more open-ended benchmarks. GSM8K provided a standardized set of grade-school word problems and became a leading testbed for multi-step arithmetic reasoning [3]. Chain-of-thought prompting encouraged models to externalize intermediate reasoning steps, while self-consistency improved answer selection by aggregating over multiple sampled reasoning paths [1], [2]. Verifier-based reranking improved answer accuracy by selecting among candidate solutions, but it treated reasoning quality as something to evaluate after generation rather than something to optimize directly during policy learning [3].

B. Preference-Based Alignment and Reward Models

Preference-based alignment changed how reward is obtained rather than eliminating the reward problem. Christiano et al. showed that pairwise human comparisons can supervise agents without access to the ground-truth reward function [5]. InstructGPT translated this into a pipeline of supervised fine-tuning, reward-model training, and PPO-based policy optimization [6]. Constitutional AI reduced reliance on direct human labels by using generated critiques [7]. DPO argued that preference optimization can recover the same target as standard RLHF without explicit online RL [8]. RewardBench made the fragility of this approach visible by showing that reward models differ in robustness over chat, reasoning, and safety tasks [11]. Self-Rewarding Language Models push the idea further by letting a language model generate its own preference-style supervision during iterative optimization [24].

C. Outcome and Process Supervision

The main reward-design distinction in reasoning tasks is between outcome supervision and process supervision. Outcome supervision evaluates the final answer as a whole, and verifier-based methods for math reasoning have shown that even coarse outcome judgments can materially improve performance [3]. Process supervision evaluates intermediate reasoning steps. Lightman et al. argue that step-level feedback produces more reliable reward models than outcome supervision alone [9]. Zhang et al. show that building strong process reward models is itself difficult: data synthesis choices and annotation quality can distort conclusions about true reasoning quality [10]. A search over reward functions can target final correctness, structural formatting, step quality, or mixtures, thereby occupying a middle ground between pure outcome rewards and fully trained process reward models.

D. Automating Reward Design

Before LLMs were used to write reward code, RL researchers had begun exploring automated objective design. Zheng et al. learned intrinsic rewards that accelerated policy-gradient training [13]. Houthoofd et al. evolved differentiable loss functions for policy learning [14]. The AutoRL literature generalized this view by treating reward design, algorithm choice, and hyperparameter tuning as automatable components of a broader RL pipeline [15]. EAGER derived auxiliary rewards from natural-language goals by asking and answering questions about the desired task [25].

Modern LLMs made automated reward design more accessible. Reward Design with Language Models used a prompted LLM as a proxy reward evaluator [16]. Language to Rewards translated language instructions into reward parameters for robotic skill synthesis [17]. Text2Reward generated dense reward programs from task descriptions, while Eureka used a coding LLM to iteratively improve reward code that in many cases matched expert-designed rewards [18], [19]. REvolve framed reward construction as an evolutionary process with populations, mutation, crossover, and selection guided by feedback [20]. CARD reduced human burden further by using dynamic feedback to improve reward code iteratively [21]. However, the strongest empirical evidence remains concentrated in robotics and control. This paper transfers the core idea of iterative reward generation and selection into mathematical reasoning for language models.

E. GRPO and Reasoning-Centered RL

GRPO replaces the separate critic used in PPO with group-relative normalization of sampled outputs and thereby reduces memory overhead while improving mathematical reasoning [22]. DeepSeek-R1 extended this trend by showing that RL can induce stronger reasoning behavior and self-correction in open models [23]. Logic-RL demonstrates that rule-based rewards and strict formatting constraints can stabilize reasoning-oriented RL even in small-data settings [26]. LoRA and related parameter-efficient fine-tuning methods play an enabling role by making repeated adaptation feasible without full-model retraining [27]. Despite this progress, most reasoning-focused RL pipelines still assume that the reward function is already known.

III. METHODS

The proposed framework consists of three major components: the GRPO reinforcement learning algorithm, a search-driven feedback strategy that uses a frontier LLM to generate and iteratively refine reward functions, and an ensemble mechanism that combines top-performing reward functions into a unified training signal. Fig. 1 provides an overview of the full pipeline.

A. Group Relative Policy Optimization

Given an input question q , GRPO samples a group of G outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$. For each output o_i , a reward r_i is computed. Under outcome supervision, the rewards are normalized by subtracting the group mean and dividing by the group standard deviation, and the resulting normalized value is assigned as the advantage of every token in the output [22]:

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})} \quad (1)$$

where $\mathbf{r} = \{r_1, r_2, \dots, r_G\}$ is the vector of group rewards. The group-relative baseline replaces the learned value function used in PPO.

The GRPO training objective maximizes the clipped surrogate loss:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{q, \{o_i\}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon) \hat{A}_i) - \beta D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right\} \right] \quad (2)$$

where $\rho_{i,t} = \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ is the probability ratio between the current and old policy, ε is the clipping parameter, and βD_{KL} regularizes against the reference policy to prevent reward hacking. Following DeepSeekMath, the KL term sits inside the per-token sum and uses the unbiased k_3 estimator $\pi_{\text{ref}}/\pi_{\theta} - \log(\pi_{\text{ref}}/\pi_{\theta}) - 1$ rather than an outer penalty [22].

The elimination of the critic network reduces memory footprint by approximately half, removes the need to train a separate value function to convergence, and avoids staleness issues that arise when a separately trained critic lags behind the improving policy. These properties make GRPO well-suited for parameter-efficient fine-tuning on limited hardware.

B. Base Model and LoRA Configuration

The base model is Meta’s Llama-3.2-3B-Instruct [28], loaded in 4-bit quantized precision using bitsandbytes. Low-Rank Adaptation (LoRA) decomposes weight updates into low-rank matrices:

$$W' = W + BA \quad (3)$$

where W is the frozen pre-trained weight matrix, and $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ are the trainable low-rank matrices with rank $r = 32$. LoRA adapters are attached to all attention and feed-forward projection matrices (q, k, v, o, gate, up, down). This results in approximately 83 million trainable parameters out of 3 billion total (2.8%). Each reward function trial saves its own LoRA checkpoint, and after evaluation the adapter is unloaded so that no learned behavior leaks between trials.

C. Dataset

The training and evaluation dataset is GSM8K, which contains 7,473 training examples and 1,319 test examples [3]. All GRPO training uses only the training split. All evaluation metrics are computed on the full 1,319-question test set.

The model is expected to produce output in a structured format:

```
<thinking>[Reasoning]</thinking>
<solution>[Final numeric answer]</solution>
```

This format separates the reasoning process from the final answer so that reward functions can independently assess reasoning quality and answer correctness.

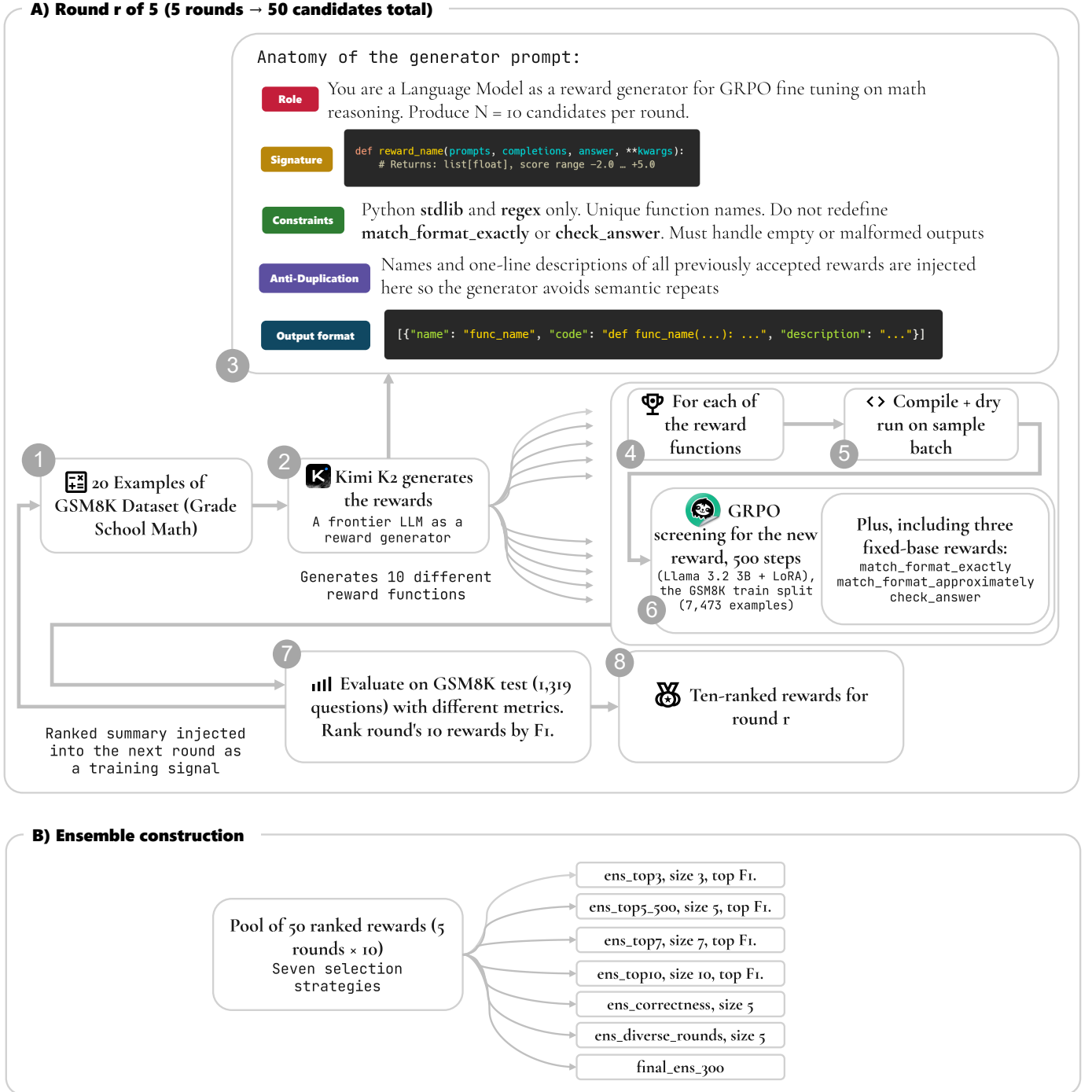


Fig. 1: Framework overview. Panel (A) details one round of search-driven reward synthesis (of five). Twenty GSM8K examples and the anatomy of the generator prompt, role, required function signature, constraints (stdlib only, no override of base rewards, graceful handling of malformed outputs), anti-duplication block that injects names and descriptions of all previously accepted rewards, and the expected JSON output format, are passed to Kimi K2, which produces ten candidate reward functions. Each candidate is compiled and dry-run, then screened for 500 GRPO steps on Llama-3.2-3B-Instruct with LoRA on the GSM8K train split alongside the three fixed base rewards, evaluated on the 1,319-question test split, and ranked by F1. The ranked summary is injected into the next round as the feedback signal. Panel (B) shows ensemble construction from the pool of 50 ranked rewards into seven selection strategies.

D. Base Reward Functions

Three fixed base reward functions are included in every training run, both during reward screening and in all ensemble configurations. They are declared in the codebase as

```
BASE_REWARD_NAMES = [
    "match_format_exactly",
    "match_format_approximately",
    "check_answer",
]
```

and are protected from being overridden or replaced by any generated reward; the four-stage sandbox rejects any candidate that attempts to redefine them. They form the always-on substrate against which discovered rewards are tested. The first, `match_format_exactly`, checks whether the output contains properly paired `<thinking>` and `<solution>` tags and awards +3.0 when matched and 0 otherwise. The second, `match_format_approximately`, provides a graduated formatting signal: +0.5 for each of the four tags present exactly once, or -1.0 for each missing or duplicated tag. The third, `check_answer`, evaluates the extracted numeric answer against ground truth with tiered scoring: +3.0 for exact match, +1.5 after whitespace normalization, +0.9 for answers in a $\pm 10\%$ ratio, +0.5 for a $\pm 20\%$ ratio, and -1.5 for wrong answers; missing or unparseable answers receive -2.5. The combined reward used during any GRPO trial of a candidate R is therefore $R_{\text{train}}(o) = R^{\text{exact}}(o) + R^{\text{approx}}(o) + R^{\text{check}}(o) + R(o)$, which means every discovered reward is always evaluated as an additive contribution on top of the three base rewards rather than as a replacement.

E. Search-Driven Reward Function Generation

The core contribution is a search-driven feedback loop that discovers reward functions beyond what the three base rewards can achieve. The process proceeds over five rounds, each generating 10 new candidate reward functions (50 total).

1) *Reward Generation via LLM*: Reward functions are generated by prompting Kimi K2 (moonshotai/kimi-k2-instruct-0905) through the Groq API. The generation prompt provides: (1) a random sample of 20 training examples, each consisting of the question and the ground-truth final answer; (2) an example reward function demonstrating the expected signature; (3) a specification requiring each function to accept prompts, completions, and answer as arguments and return a list of floats; and (4) a constraint that function names must not duplicate previously generated ones nor any of `match_format_exactly`, `match_format_approximately`, or `check_answer` (the protected base rewards from Section III-D). Generation uses temperature 0.6 with a maximum of 8,192 output tokens. The model returns a JSON array of name, code, and description fields. Each candidate that passes validation is then trained with GRPO on top of the three fixed base rewards for 500 steps using the hyperparameters described in Section III-G, and evaluated on the GSM8K test set to obtain the F1, accuracy, precision, and recall used to rank rewards within and across rounds.

2) *Feedback Loop*: After each round, all previously evaluated reward functions are ranked by F1 score on the test set. A ranked summary of these results is passed back to Kimi K2 in the prompt for the next round, explicitly instructing the model to avoid duplicating names or semantics and to focus on uncovered aspects of mathematical reasoning. This ranked-feedback mechanism supplies implicit selection pressure: high-scoring rewards survive as descriptive information that conditions the next round of generation, while lower-scoring ones implicitly signal directions to avoid. The signal re-entering the next round is a compact textual summary of the form “name=<reward>, round=<r>, F1=<value>, acc=<value>”, and the search space itself has no learnable parameters.

3) *Validation and Screening*: Each generated reward function passes through a four-stage validation pipeline: (1) an AST pass rejects imports outside a small standard-library whitelist; (2) the code is executed in a locked-down environment with stripped `__builtins__`; (3) the function is rebuilt in a restricted globals namespace exposing only `re`, `math`, and the tag constants; and (4) a probe call with dummy inputs verifies that the function returns a `list[float]` of the correct length. Rewards that fail any stage are discarded. Valid rewards are screened through 500-step GRPO training runs with the three base rewards always included alongside the candidate. The entire search takes approximately 40 hours on a single NVIDIA RTX 5090 GPU.

F. Ensemble Reward Construction

After the search is complete and all 50 individual rewards have been ranked, top-performing rewards are combined into ensembles. Each ensemble always retains the three base rewards alongside K discovered rewards and sums all scores element-wise:

$$R_{\text{ensemble}}(o) = \sum_{b=1}^3 R_b^{\text{base}}(o) + \sum_{k=1}^K R_k(o) \quad (4)$$

In the configurations reported below, the count of rewards (e.g., “top 3” or “top 5”) refers to the number of discovered rewards added on top of the three fixed base rewards. Seven ensemble configurations were evaluated. These configurations vary in size (3 to 10 discovered rewards) and selection strategy (ranking-based vs. diversity-based). The diversity-focused ensemble, `ens_diverse_rounds`, selects one discovered reward from each of the five rounds to maximize coverage of different reward strategies.

Algorithm 1 summarizes the complete framework, including the four-stage sandbox validation used to screen every generated reward function before it enters GRPO training.

G. Training Configuration

GRPO training uses a group size of $G = 4$. The effective batch size is 4 (per-device batch size 1, gradient accumulation 4). The learning rate is 5×10^{-6} with cosine annealing, warmup ratio 0.1, and weight decay 0.1. The optimizer is AdamW with 8-bit quantization. Maximum gradient norm is

Algorithm 1 Search-Driven Reward Function Synthesis

Require: Base policy π_θ (Llama-3.2-3B + LoRA); dataset $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}$ (GSM8K); frontier LLM generator $\mathcal{G}$ (Kimi K2); base rewards $\mathcal{B} = \{R_1^{\text{base}}, R_2^{\text{base}}, R_3^{\text{base}}\}$; rounds $N=5$; rewards per round $M=10$; GRPO steps $T=500$

Ensure: Ranked rewards $\mathcal{P}$; best ensemble E^*

```

1:  $\mathcal{P} \leftarrow \emptyset; H \leftarrow \emptyset$  ▷ pool and history
2: for  $r = 1$  to  $N$  do
3:    $\text{prompt}_r \leftarrow \text{BUILDPROMPT}(H, \text{samples from } \mathcal{D}_{\text{train}})$ 
4:    $\mathcal{C}_r \leftarrow \mathcal{G}(\text{prompt}_r)$  ▷  $M$  candidate reward sources
5:   for all  $R \in \mathcal{C}_r$  do
6:      $s_1 \leftarrow \text{CHECKMODULES}(R)$  ▷ Stage 1: imports
7:      $s_2 \leftarrow \text{BUILDLOCKEDDOWNFN}(R)$  ▷ Stage 2: AST sandbox
8:      $s_3 \leftarrow \text{BUILDSAFECALLABLE}(R)$  ▷ Stage 3: wrap & resource limits
9:      $s_4 \leftarrow \text{PROBECALLABLE}(R)$  ▷ Stage 4: dry-run on batch
10:    if not  $(s_1 \wedge s_2 \wedge s_3 \wedge s_4)$  then
11:      continue ▷ reject unsafe/invalid reward
12:    end if
13:     $\mathcal{R}_{\text{train}} \leftarrow \mathcal{B} \cup \{R\}$ 
14:     $\pi_R \leftarrow \text{GRPO}(\pi_\theta, \mathcal{D}_{\text{train}}, \mathcal{R}_{\text{train}}, T)$ 
15:     $(\text{acc}, P, \text{rec}, F_1) \leftarrow \text{EVALUATE}(\pi_R, \mathcal{D}_{\text{test}})$ 
16:     $\mathcal{P} \leftarrow \mathcal{P} \cup \{(R, F_1, r)\}$ 
17:  end for
18:   $H \leftarrow \text{RANKSUMMARY}(\mathcal{P})$  ▷ selection pressure
19: end for

20: Ensemble stage:
21:  $\mathcal{E} \leftarrow \text{BUILDENSEMBLECONFIGS}(\mathcal{P})$  ▷ 7 configs, ranking & diversity
22: for all  $E \in \mathcal{E}$  with discovered rewards  $\{R_k\}_{k=1}^K$  do
23:    $R_E(o) \leftarrow \sum_{b=1}^3 R_b^{\text{base}}(o) + \sum_{k=1}^K R_k(o)$ 
24:    $\pi_E \leftarrow \text{GRPO}(\pi_\theta, \mathcal{D}_{\text{train}}, \{R_E\}, T)$ 
25:    $F_1^E \leftarrow \text{EVALUATE}(\pi_E, \mathcal{D}_{\text{test}})$ 
26: end for
27:  $E^* \leftarrow \arg \max_{E \in \mathcal{E}} F_1^E$ 
28: return  $\mathcal{P}, E^*$ 

```

1.0. Individual screening trials run for 500 steps, and ensemble training also uses 500 steps (except an initial pilot at 300 steps). The 500-step screening budget follows the recommendation in the official Unsloth GRPO documentation [29], which suggests training for at least 300 steps to obtain a reliable reward signal; we exceed this threshold by 67% to balance ranking reliability against the wall-clock cost of evaluating 50 candidate rewards. The training pipeline is built on the TRL library [30] with Unsloth [31] for efficient 4-bit LoRA fine-tuning.

H. Evaluation

All evaluations use greedy decoding (temperature = 0). The model output is parsed to extract the content between `<solution>` tags. Four metrics are computed over the

1,319-question test set: precision ($\text{TP} / (\text{TP} + \text{FP})$), recall ($\text{TP} / (\text{TP} + \text{FN})$), F1 (harmonic mean of precision and recall), and accuracy (the fraction of the 1,319 test questions for which the policy produces an extracted answer that matches the ground truth). In this setting TP counts questions whose extracted answer matches the ground truth, FP counts questions whose extracted answer does not match, and FN counts questions that produce no parseable `<solution>` answer. Outputs that fail strict-format extraction are therefore counted as wrong rather than skipped, so the reported accuracy is the standard exact-match rate under strict tag-format parsing. To isolate the contribution of strict format parsing from reasoning quality, supplementary Section S6 additionally reports exact-match accuracy under flexible parsing, in which a numeric-token regex over the full completion is used as a fallback when strict extraction fails. F1 is the primary ranking metric because it balances answer correctness against format compliance. Bootstrap 95% confidence intervals (10,000 resamples with replacement over the 1,319 test items) are reported for all headline accuracies, and pairwise McNemar tests are reported for the seven ensemble configurations. Because 21 pairwise tests are performed, the corresponding Bonferroni-corrected significance threshold is $\alpha = 0.05/21 \approx 0.0024$.

Five off-the-shelf models of comparable size (2B–4B parameters) are evaluated as baselines on the same test set under identical conditions: Qwen 3/3.5 (4B, 2B) and Phi-4-Mini (3.8B, reasoning variant).

The evaluation protocol is designed so that the contribution of the discovered rewards can be read off by comparison rather than asserted in isolation. Two controls are evaluated under identical decoding, parsing, and bootstrap procedures: a base-rewards-only configuration, which isolates the effect of adding any discovered reward on top of the three protected base rewards, and a random-5 configuration, which isolates the effect of ranked selection from the additive effect of merely combining five rewards. The headline ensemble is additionally retrained under three independent GRPO seeds (Section IV-C2) to characterize seed-induced variance. Because all configurations — discovered, base-only, random, and multi-seed — are evaluated under the same protocol, the comparisons reported in Section IV are internal to the pipeline and the relative ordering across configurations is the primary object of inference.

IV. RESULTS

A. Search Trajectory

The search over five rounds produced 50 candidate reward functions. The F1 scores of all 50 rewards (sorted from highest to lowest, supplementary Fig. S1) reveal a steep drop-off: the top 6 rewards achieve F1 scores above 0.75, while the majority cluster between 0.55 and 0.65. The worst reward achieves an F1 of only 0.059. Only approximately 12% of rewards exceed 0.60 accuracy.

The top five individual rewards are: `thinking_steps_count` (F1=0.787, Round 5), `thinking_has_calc` (F1=0.786, Round 5), `numeric_reasoning_depth` (F1=0.783, Round 4),

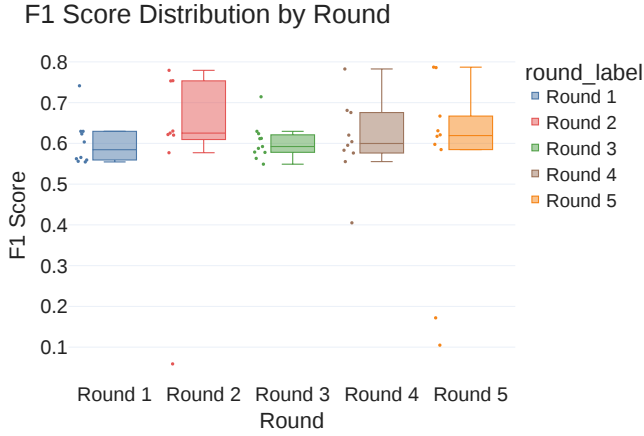


Fig. 2: Box plot of F1 scores by round. The median F1 increases from Round 1 through Round 5, though with a dip in Round 3.

`step_by_step_accuracy` (F1=0.779, Round 2), and `reasoning_completeness` (F1=0.754, Round 2). Three of these five focus on the quality and structure of the chain-of-thought reasoning; the other two evaluate completeness and step-by-step correctness. We caution that these are unverified structural rewards and could in principle be exploitable by a policy that only pads its reasoning block; we audit this concern explicitly in Section IV-D. For the same reason these rewards are never used alone: they are summed with the three correctness-checking base rewards. The full F1 ranking of all 50 rewards is given in the supplementary material (Fig. S1).

1) *Performance Variation Over Rounds*: Fig. 2 presents box plots of F1 scores grouped by round. The mean F1 increases from 0.596 in Round 1 to 0.632 in Round 5. The improvement is not strictly monotonic: Round 3 dips to 0.604 before recovery in Round 4 (0.618) and Round 5. This non-monotonic pattern may reflect the search exploring a different region of reward function space that did not yield strong rewards on average, even as the system continued accumulating information about effective strategies.

A precision–recall scatter over all 50 rewards (Fig. 3) shows that most rewards cluster where recall is high (above 0.8) but precision is moderate (0.5–0.7), indicating they encourage the model to always produce a formatted answer without strongly discriminating correct from incorrect; the top-performing rewards achieve a better balance with both metrics above 0.7.

B. Analysis of Top-Performing Reward Functions

1) *Reward Function Code*: Listing 1 shows the top-ranked reward function. It counts distinct non-empty lines in the `<thinking>` block and assigns graduated scores of +3, +2, or +1 for three, two, or one reasoning steps respectively, and −1 for a missing thinking block. This reward does not check answer correctness; the three base rewards provide that signal.

The second-ranked reward, `thinking_has_calc` (F1=0.786, Round 5), applies a binary check for whether the thinking section contains arithmetic operations using the regex pattern `[0-9]+\s*[\+|\-|\*|/]`. A score of +2 is awarded

Precision vs Recall (size = F1)

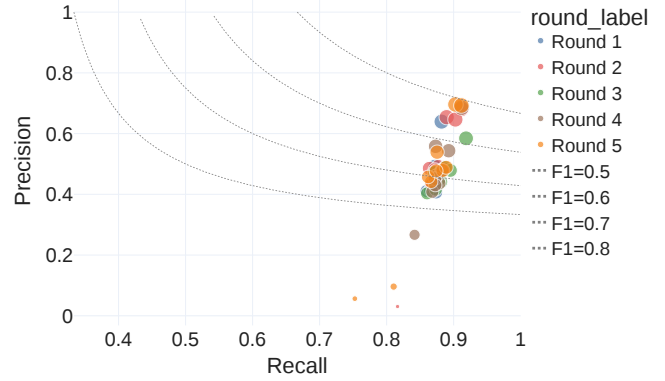


Fig. 3: Precision versus recall for all 50 individual reward functions, colored by round.

Listing 1: Top-ranked reward: `thinking_steps_count` (F1=0.787, Round 5). Rewards multi-line reasoning structure without checking correctness.

```
1 def thinking_steps_count(prompts, completions,
2   answer, **kwargs):
3   import re
4   scores = []
5   for completion, _ in zip(completions, answer):
6   resp = completion[0]['content']
7   m = re.search(
8     r'<thinking>(.*?)</thinking>',
9     resp, re.DOTALL)
10  if not m:
11    scores.append(-1.0)
12    continue
13  steps = [s for s in
14    re.split(r'\n+', m.group(1).strip())
15    if s.strip()]
16  if len(steps) >= 3: scores.append(3.0)
17  elif len(steps) == 2: scores.append(2.0)
18  elif len(steps) == 1: scores.append(1.0)
19  else: scores.append(-1.0)
20  return scores
```

when explicit calculations are present and 0 otherwise, with −1 for a missing thinking block. The highest-ranked hybrid reward, `step_by_step_accuracy` (F1=0.779, Round 2), combines arithmetic-step detection with answer correctness and assigns graduated scores from +4 to −1; its full source is given in the supplementary material (Listing S2).

The top ten discovered rewards group into three categories: pure-process (4 rewards, mean F1 = 0.759), hybrid process plus outcome (4 rewards, mean F1 = 0.757), and pure-outcome (2 rewards, mean F1 = 0.695). The pure-process rewards achieve the highest mean F1 because the three base rewards already provide strong correctness signals; adding redundant outcome verification yields diminishing returns, while process-oriented signals such as step counts, arithmetic detection, and numeric density are complementary. The full taxonomy table is given in the supplementary material (Table S1).

2) *Empirical Patterns and Their Limits*: Three patterns appear in the top ten rewards. The top three discovered rewards (`thinking_steps_count`, `thinking_has_calc`, `numeric_reasoning_depth`) all evaluate reasoning

structure rather than re-checking the final answer, with mean F1 0.785 versus 0.695 for the two pure outcome rewards in the top ten; we frame this complementarity as suggestive given the small sample (3 vs. 2). Eight of the top ten use three or more distinct score levels rather than binary $\{0, 1\}$, consistent with the known benefits of graduated reward shaping for credit assignment. Code length is not predictive: over the full pool of 50 the correlation between line count and F1 is $r = 0.04$, and the bottom-ten mean line count (20.5) is comparable to the top ten (24.0).

Three discovered rewards in the pool of 50 yielded catastrophic GRPO performance (F1 well below the 0.609 base-only baseline): `efficiency_vs_accuracy` (F1=0.059), `thinking_no_answer_leak` (F1=0.105), and `thinking_length_range` (F1=0.172). The common thread is that aggressive penalty terms with no correctness gate dominate the gradient before the policy has discovered any high-reward region of behavior, a known failure mode of reward shaping in sparse-reward RL [32]. This is one practical reason to keep the three base rewards on at all times. A full per-reward post-mortem is given in the supplementary material (Section S3).

C. Ensemble Results

Table I summarizes the performance of all ensemble configurations together with bootstrap 95% confidence intervals. The numerically best ensemble is `ens_diverse_rounds`, which combines the three base rewards with five discovered rewards drawn one per round. It achieves F1=0.795 and accuracy=0.660 (95% CI [0.635, 0.686]). However, the bootstrap intervals of the five largest ensembles (5–10 discovered rewards) all overlap substantially. Two control rows sit below the main results: “Base only (3 rewards)” is GRPO with only the three protected base rewards under identical hyperparameters and seed and isolates the contribution of the discovered rewards; “`ens_random5`” replaces ranked selection with five rewards drawn uniformly at random from the pool of 50 and isolates the contribution of selection from that of merely adding five rewards. The random draw included `efficiency_vs_accuracy`, one of the catastrophic rewards (F1=0.059 in isolation), which collapses training to F1=0.047 even with the three base rewards present.

1) *Statistical Significance*: Pairwise McNemar tests over the per-question correctness vectors of the seven ensembles produce 21 comparisons, requiring a Bonferroni-corrected significance threshold of $\alpha = 0.0024$. The five large ensembles (≥ 5 discovered rewards trained for 500 steps) are not statistically distinguishable from one another at this threshold (p -values between 0.04 and 0.39 for all pairs). Both the three-reward configuration and the five-reward 300-step pilot are significantly worse than every five-reward-or-larger ensemble ($p < 10^{-6}$ for the relevant pairs). The full p -value table is provided in the supplementary material (Table S2).

2) *Robustness Over Seeds*: To probe whether the reported ranking is sensitive to the GRPO training seed, the `ens_diverse_rounds` configuration was retrained from scratch under two additional seeds (1234 and 2025) on the

TABLE I: Ensemble performance on GSM8K (1,319 questions). “#R” counts discovered rewards added to the three fixed base rewards. “Accuracy” is exact-match accuracy under strict tag-format parsing, defined in Section III-H; 95% bootstrap CI in brackets. The five large ensembles (≥ 5 discovered rewards) are statistically indistinguishable under McNemar with Bonferroni correction (supplementary Table S2).

Configuration	# R	F1	Accuracy [95% CI]	Rec.
<code>ens_diverse_rounds</code>	5	0.795	0.660 [0.635, 0.686]	0.892
<code>ens_top5_500</code>	5	0.789	0.651 [0.626, 0.677]	0.909
<code>ens_top10</code>	10	0.787	0.648 [0.622, 0.673]	0.911
<code>ens_correctness</code>	5	0.784	0.644 [0.619, 0.670]	0.883
<code>ens_top7</code>	7	0.782	0.641 [0.616, 0.667]	0.914
<code>ens_top3</code>	3	0.738	0.585 [0.558, 0.611]	0.759
<code>final_ens_300</code>	5	0.743	0.591 [0.564, 0.617]	0.896
Best Indiv.	1	0.787	0.649 [0.624, 0.674]	;
<code>ens_random5</code>	5	0.047	0.024 [0.017, 0.033]	0.762
Base only (3 rewards)	0	0.609	0.438 [0.412, 0.466]	;

same fixed Llama-3.2-3B base, with identical hyperparameters and an unchanged 1,319-question evaluation set. Across the three seeds, F1 is 0.785 ± 0.013 (range 0.771 to 0.795) and accuracy is 0.646 ± 0.017 . Two observations follow. First, the diversity ensemble’s lead over the top-5 ranking ensemble (default-seed gap of $0.795 - 0.789 = 0.006$) is well inside the seed-to-seed variation, and one of the three seeds (2025, F1=0.771) trails the default-seed top-5 ensemble. The diversity-vs-ranking comparison is therefore reported as exploratory rather than confirmatory throughout the paper. Second, even the worst seed of the diversity ensemble (F1=0.771) substantially exceeds both the three-reward configuration (F1=0.738) and the base-rewards-only baseline (F1=0.609) by margins much larger than the seed-to-seed spread. The qualitative ordering “base only < three-reward < five large ensembles” is therefore robust to seed choice, while finer-grained ranking among the five large ensembles is not. Per-seed numbers are tabulated in the supplementary material (Table S3).

A visualization of all ensemble configurations on the four evaluation metrics is shown in Fig. 4.

D. Reward-Hacking and Format-Penalty Audits

Three of the top five discovered rewards count structural features of the `<thinking>` block without verifying answer correctness, with `thinking_steps_count` (F1=0.787) the most aggressive: it pays +3.0 for at least three newline-separated lines and is the textbook target for length-padding hacks. On the 1,319 GSM8K test items the trained policy emits a mean of 6.13 lines inside `<thinking>` (median 6), so the ≥ 3 threshold is exceeded organically rather than minimally. Splitting outputs by correctness, correct responses contain 6.47 lines on average and incorrect ones 5.51, with arithmetic-step-pattern density 0.63 vs. 0.51 respectively. If the model were padding empty lines, the correct vs. incorrect distributions would be similar or inverted; instead, correct answers are accompanied by both more lines and a higher density of arithmetic content. The simplest line-padding hypothesis is

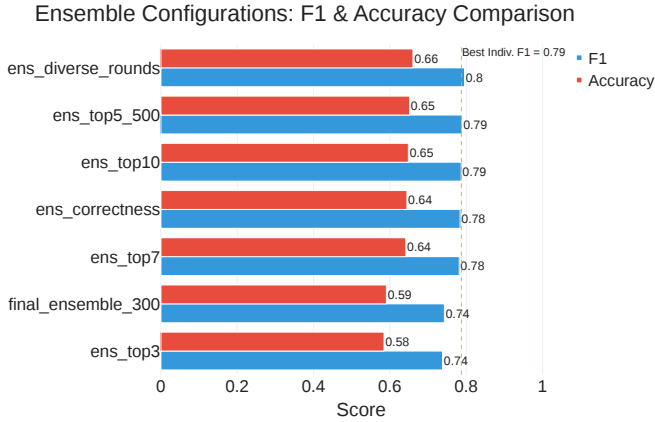


Fig. 4: All ensemble configurations compared on four evaluation metrics. The five large ensembles cluster tightly; only the three-reward and 300-step configurations fall out of the band.

TABLE II: Off-the-shelf model performance on GSM8K vs. GRPO-trained model. “Accuracy” for the GRPO-trained model is the strict tag-format exact-match accuracy of Section III-H; for the off-the-shelf baselines it equals their native (already flexible) numeric-extraction accuracy. “Acc. (flex)” for the GRPO model uses flexible parsing (Section IV-D; full table in supplementary Section S6).

Model	Params	Accuracy	Acc. (flex)	F1
qwen3:4b	4B	0.634	;	0.776
phi4-mini:3.8b	3.8B	0.629	;	0.773
qwen3.5:4b	4B	0.449	;	0.620
phi4-mini-reas.	3.8B	0.409	;	0.581
qwen3.5:2b	2B	0.403	;	0.575
GRPO (ours)	3B	0.660	0.673	0.795

therefore not supported by the per-question evidence.

A separate concern is the strict tag-format penalty in the accuracy column. Re-evaluating the trained policies under flexible answer extraction (numeric-token regex over the full completion as fallback) shifts accuracy by +0.012 to +0.020 for the five large ensembles and by +0.092 for the three-reward ensemble. The format penalty for the large ensembles is therefore small; the three-reward configuration is producing correct reasoning that frequently lands in a non-canonical solution tag. Full per-configuration numbers and a longer audit discussion are provided in the supplementary material (Sections S6 and S7).

E. Baseline Comparison

Table II presents the performance of five off-the-shelf models in the 2–4B parameter band. The trained Llama-3.2-3B model achieves the highest F1 (0.795) and accuracy (0.660) among models in this size range, surpassing all evaluated baselines. The remaining baselines span an accuracy range from 0.403 to 0.634.

A direct read of Table II is that the GRPO-trained Llama-3.2-3B leads its size band on both strict and flexible accuracy and on F1.

The contribution of this paper is methodological rather than a claim to a new state of the art on GSM8K. The question is whether the reward specification used during RL post-training can be treated as a reliably searchable object. The Llama-3.2-3B base was chosen because a full search over 50 rewards plus seven ensembles plus three reproducibility seeds fits inside an academic single-GPU budget; the same protocol applies unchanged to any larger base, and wall-clock cost grows roughly linearly with model size. The evidence that matters is internal to the pipeline: the F1 of generated rewards rises from round to round (Fig. 2); the random-5 control collapses to F1=0.047 while ranked ensembles cluster near 0.79; the qualitative ordering “base only < three-reward < five large ensembles” holds over three independent training seeds. Re-running the pipeline on a stronger base to test whether the discovered rewards stack on top of a higher starting point is a natural item of future work (Section VI-A).

F. Computational Cost

The full search over 5 rounds and 50 rewards takes approximately 40 hours of wall-clock time on a single NVIDIA RTX 5090 GPU (32 GB VRAM); per-trial timing details are reported in supplementary Section S8.

V. DISCUSSION

The results support three main conclusions about search-driven reward optimization for LLM reasoning.

First, iterative LLM-guided generation conditioned on ranked feedback yields measurable improvement over rounds. The mean F1 of generated rewards increased from 0.596 in Round 1 to 0.632 in Round 5, and the three highest-scoring individual rewards all emerged from the later rounds. The non-monotonic dip in Round 3 suggests that the search occasionally explores qualitatively different regions of reward space, which is desirable to prevent premature convergence on a single strategy.

Second, combining several discovered rewards into ensembles produces stronger training signals than the best single reward. All five large ensembles (5 to 10 discovered rewards added to the three base rewards, 500 steps) cluster between F1 0.78 and 0.80 and accuracy 0.64 to 0.66, and pairwise McNemar tests with Bonferroni correction show that they are not statistically distinguishable from one another. The three-reward configuration and the 300-step pilot fall significantly below this picture. A three-seed re-training of the best ensemble yields F1 of 0.785 ± 0.013 (Section IV-C2); the original 0.795 vs. 0.789 gap therefore sits inside seed-to-seed variation. A random-5 control drawn from the same pool collapses to F1=0.047, far below the base-only baseline; the ranked feedback loop, not the mere act of stacking five rewards, is what produces the gain.

Third, the discovered rewards are largely structural and could in principle be hackable, but the audit in Section IV-D shows that the most exploitable reward (`thinking_steps_count`) does not, in practice, induce empty line padding under this base model and budget; correct

answers carry both more lines and a higher density of arithmetic content than incorrect ones. Combined with the format-penalty analysis in supplementary Section S6, which shows that strict tagging accounts for only $\sim 1\text{--}2\%$ of the absolute accuracy of the large ensembles, the discovered rewards behave as expected complements to the three base correctness rewards rather than as independent gameable surrogates.

A. Limitations

A number of design choices in this work define a clear scope for the present study. The focused setup, in which every component of the pipeline other than the reward specification is held fixed, makes the results reproducible at the scale of an academic compute budget [33] and isolates the effect of changing the reward specification on downstream reasoning performance. Mapping how the discovered rewards behave when the benchmark, base model, generator, or optimizer changes is a natural extension that we leave to follow-up studies (Section VI-A).

The screening budget of 500 GRPO steps per candidate exceeds the 300-step minimum recommended by the Unsloth GRPO documentation [29] and is appropriate for ranking 50 candidates on a single GPU, but rewards whose effects only emerge after substantially longer training cannot be separated at this budget; re-training the top-ranked rewards for 2,000 steps is the cleanest way to test how stable the 500-step ranking is asymptotically. Multi-seed evidence is currently reported for the diversity-selected ensemble only (Section IV-C2), which suffices to show that the qualitative ordering “base only < three-reward < five large ensembles” is robust to seed but that finer-grained ranking among the five large ensembles is not. The ensemble mechanism is also intentionally simple: rewards are summed with equal weights, and learned weighting is left as a complementary direction.

A more structural limitation concerns the transferability of the discovered reward code itself. The 50 reward functions are written against the `<thinking>/<solution>` tag schema and GSM8K-style numeric final answers; evaluating them on a different schema or on benchmarks with non-numeric answers requires rewriting the regex and answer-comparison logic.

VI. CONCLUSION

This paper investigated whether the reward function used in reinforcement learning for language-model reasoning can be treated as an object of search rather than a fixed design choice. A search-driven reward synthesis framework was developed and evaluated on the GSM8K mathematical reasoning benchmark over five rounds. The search produced 50 candidate rewards and seven ensemble configurations.

The search yielded measurable improvement over rounds, with the top-performing rewards emerging from later rounds. Combining several discovered rewards into ensembles produced stronger training signals than any single reward; under multiple-comparison correction, the five large ensembles are statistically indistinguishable from one another, while the three-reward and 300-step configurations are significantly

worse. The diversity-vs-ranking comparison is therefore exploratory rather than confirmatory. A reward-hacking audit on the most exploitable structural reward shows that, in practice, correct responses contain more reasoning steps and a higher density of arithmetic operations than incorrect ones, which is the opposite of the line-padding failure mode predicted by a naive reading of the reward.

A. Future Work

Future directions include applying the same pipeline to other base models of similar or larger scale (e.g., the Qwen-3, Phi-4, Mistral, Llama-3, and Gemma 3 families spanning 2B to 8B parameters) to test whether the discovered rewards stack on top of different starting points; comparing alternative reward generators (GPT, Claude, Gemini, DeepSeek) against Kimi K2 to characterize how generator choice influences which rewards are discovered; comparing alternative policy optimizers (PPO, RLOO, DPO) against GRPO; population-based strategies with explicit crossover, mutation, and diversity maintenance; multi-seed runs (3–5 seeds per configuration) on every ensemble; longer-horizon screening (e.g., 2,000 GRPO steps for the top five rewards); held-out reasoning benchmarks beyond GSM8K (e.g., MATH, AIME, MMLU-STEM, BBH); and learned ensemble weighting in place of equal-weight summation.

Reproducibility

All code, generated reward functions, and per-question correctness vectors used to compute the bootstrap CIs and McNemar tests are released at <https://github.com/INQUIRELAB/search-reward-rl>. The release sandboxes external API keys behind environment variables, ships the four-stage validation pipeline, and includes scripts to regenerate every table and figure in this paper. Extended results (full reward-taxonomy table, McNemar p-value matrix, three-seed table, strict-vs-flexible-parse comparison, long-form hacking audit, full F1 ranking, full source of the top hybrid reward, and per-trial timing) are in the supplementary material.

REFERENCES

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.
- [2] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *arXiv preprint arXiv:2203.11171*, 2022.
- [3] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [5] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *Advances in neural information processing systems*, vol. 30, 2017.
- [6] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, *et al.*, “Training language models to follow instructions with human feedback,” *Advances in neural information processing systems*, vol. 35, pp. 27730–27744, 2022.

- [7] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, *et al.*, “Constitutional ai: Harmlessness from ai feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [8] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in neural information processing systems*, vol. 36, pp. 53728–53741, 2023.
- [9] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” in *The twelfth international conference on learning representations*, 2023.
- [10] Z. Zhang, C. Zheng, Y. Wu, B. Zhang, R. Lin, B. Yu, D. Liu, J. Zhou, and J. Lin, “The lessons of developing process reward models in mathematical reasoning,” in *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 10495–10516, 2025.
- [11] N. Lambert, V. Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, *et al.*, “Rewardbench: Evaluating reward models for language modeling, 2024,” *URL https://arxiv.org/abs/2403.13787*, vol. 40.
- [12] D. Hadfield-Menell, S. Milli, P. Abbeel, S. J. Russell, and A. Dragan, “Inverse reward design,” *Advances in neural information processing systems*, vol. 30, 2017.
- [13] Z. Zheng, J. Oh, and S. Singh, “On learning intrinsic rewards for policy gradient methods,” *Advances in neural information processing systems*, vol. 31, 2018.
- [14] R. Houthoofd, Y. Chen, P. Isola, B. Stadie, F. Wolski, O. Jonathan Ho, and P. Abbeel, “Evolved policy gradients,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [15] J. Parker-Holder, R. Rajan, X. Song, A. Biedenkapp, Y. Miao, T. Eimer, B. Zhang, V. Nguyen, R. Calandra, A. Faust, *et al.*, “Automated reinforcement learning (autorl): A survey and open problems,” *Journal of Artificial Intelligence Research*, vol. 74, pp. 517–568, 2022.
- [16] M. Kwon, S. M. Xie, K. Bullard, and D. Sadigh, “Reward design with language models,” *arXiv preprint arXiv:2303.00001*, 2023.
- [17] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, *et al.*, “Language to rewards for robotic skill synthesis,” *arXiv preprint arXiv:2306.08647*, 2023.
- [18] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, and T. Yu, “Text2reward: Reward shaping with language models for reinforcement learning,” *arXiv preprint arXiv:2309.11489*, 2023.
- [19] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar, “Eureka: Human-level reward design via coding large language models,” *arXiv preprint arXiv:2310.12931*, 2023.
- [20] R. Hazra, A. Sygkounas, A. Persson, A. Loutfi, and P. Z. D. Martires, “Revolve: Reward evolution with large language models using human feedback,” *arXiv preprint arXiv:2406.01309*, 2024.
- [21] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, and X. Li, “A large language model-driven reward design framework via dynamic feedback for reinforcement learning,” *Knowledge-Based Systems*, vol. 326, p. 114065, 2025.
- [22] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, *et al.*, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [23] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, *et al.*, “Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [24] W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. Weston, “Self-rewarding language models,” *arXiv preprint arXiv:2401.10020*, 2024.
- [25] T. Carta, P.-Y. Oudeyer, O. Sigaud, and S. Lamprier, “Eager: Asking and answering questions for automatic reward shaping in language-guided rl,” *Advances in neural information processing systems*, vol. 35, pp. 12478–12490, 2022.
- [26] T. Xie, Z. Gao, Q. Ren, H. Luo, Y. Hong, B. Dai, J. Zhou, K. Qiu, Z. Wu, and C. Luo, “Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning,” *arXiv preprint arXiv:2502.14768*, 2025.
- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, *et al.*, “Lora: Low-rank adaptation of large language models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [28] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [29] Unsloth AI, “Reinforcement learning (rl) guide,” <https://unsloth.ai/docs/get-started/reinforcement-learning-rl-guide>, 2025. Documentation accessed 2026.
- [30] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, S. Huang, K. Rasul, and Q. Gallouédec, “Trl: Transformers reinforcement learning,” 2020.
- [31] U. AI, D. Han-Chen, and M. Han-Chen, “Unsloth,” <https://github.com/unslothai/unsloth>, 2025.
- [32] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *ICML*, vol. 99, pp. 278–287, Citeseer, 1999.
- [33] A. Khandelwal, T. Yun, N. V. Nayak, J. Merullo, S. H. Bach, C. Sun, and E. Pavlick, “\$100 k or 100 days: Trade-offs when pre-training with academic resources,” *arXiv preprint arXiv:2410.23261*, 2024.



Arash Ahmadi received the B.S. degree in computer engineering from the University of Kurdistan in 2023. He is currently pursuing the M.S. and Ph.D. degrees in electrical and computer engineering at the University of Oklahoma, where he is a Graduate Research Assistant in the INQUIRE Laboratory. His research focuses on reinforcement learning, parameter-efficient fine-tuning of language models, and deployment of small language models on resource-constrained edge devices for agentic systems.



Sarah S. Sharif (Senior Member, IEEE) is an assistant professor in the School of Electrical and Computer Engineering at the University of Oklahoma. She holds a Ph.D. in Electrical Engineering and two M.Sc. degrees in Physics and Electrical Engineering, with 10+ years of industrial R&D experience. Her work in INQUIRE focuses on next-generation computing and sensing through optical, quantum-optical, and neuromorphic devices.



Yaser M. Banad (Senior Member, IEEE) is an Associate Professor in the School of Electrical and Computer Engineering at the University of Oklahoma. He received the M.Sc. and Ph.D. degrees from Louisiana State University in 2016. An author of 100+ peer-reviewed publications, his research spans neuromorphic computing, energy-efficient devices, neural-inspired AI acceleration, and semiconductor materials.